

Summary: Pandas-II

1. Focus points

- Learn **DataFrame attributes** (metadata and structure).
 - Explore **methods** for summarizing and inspecting data.
 - Perform **row and column operations** (add, delete, rename).
 - Gain practical coding insights from Jupyter Notebook examples.
-

2. Attributes of a DataFrame

Attributes describe the structure and metadata of a DataFrame.

- **.index** → Row labels.
Example: `RangeIndex(start=0, stop=1000, step=1)` → 1000 rows labeled 0–999.
 - **.shape** → Dimensions (rows, columns).
Example: `(1000, 11)` → 1000 rows, 11 columns.
 - **.columns** → Column labels.
Example: `Index(['id', 'first_name', 'last_name', 'email', 'gender', ...])`.
 - **.dtypes** → Data types of each column (`int64`, `float64`, `object`).
 - Jupyter tip: You can check a single column's dtype:
 - `df['gender'].dtypes`
 - **.values** → 2D NumPy array of data.
Example: `df.values` returns raw array form.
 - **.size** → Total elements = rows × columns.
Example: 11000.
 - **.empty** → Boolean, True if DataFrame has no data.
 - **.ndim** → Number of dimensions (always 2 for DataFrame).
 - **.hasnans** → Checks for missing values.
-

3. Common Methods

- **.info()** → Concise summary (rows, columns, non-null counts, dtypes, memory usage).
Example: `df.info()`.
- **.head(n)** → First `n` rows (default 5).
Example: `df.head(3)` shows first 3 rows.
- **.tail(n)** → Last `n` rows (default 5).
Example: `df.tail(3)` shows last 3 rows.
- **.describe()** → Statistical summary (count, mean, std, min, quartiles, max).
Example: `df.describe()`.

- `.transpose()` → Swap rows and columns.

Example: `df.transpose()`.

💡 **Notebook Tip:** Outputs in Jupyter show truncated tables, but you can use `df.head()` or `df.tail()` to control how much data is displayed.

4. Operations on Rows and Columns

A) Adding Columns

- **Scalar value:**
- `df['Country'] = 'USA'`

Adds "Country" column with "USA" for all rows.

- **Based on other columns:**
- `df['full_name'] = df['first_name'] + ' ' + df['last_name']`
- **Insert at custom position:**
- `full_name = df['first_name'] + ' ' + df['last_name']`
- `df.insert(3, 'full_name2', full_name)`

Adds column at index position 3.

⚠️ **Tip from Notebook:** If you try to assign a list of values shorter than the DataFrame length, you'll get a `ValueError`.

B) Adding Rows

- Using `.loc` with a new index:
- `df.loc['1000'] = [1001, 'Sachin', 'Khote', 'Sachin Khote',
'sachin@deloitte.com', 'Male', None, None,
None, 'California', 'California', 92127.0,
'USA', 'Sachin Khote']`
- Adds a new row at index 1000.
- ⚠️ If index already exists, it overwrites that row.

💡 **Notebook Warning:** Pandas shows a **FutureWarning** about concatenation with empty/NA entries. This means future versions may handle dtype inference differently.

C) Deleting Rows

- Drop by index:
- `df.drop(index='1000', inplace=True)`

- Removes row with index 1000.
-

D) Deleting Columns

- Drop multiple columns:
 - `columns_to_drop = ['street_num', 'street_name', 'street_suffix']`
 - `df.drop(columns=columns_to_drop, inplace=True)`
 - Removes specified columns.
-

E) Renaming Row Labels

- Rename single index:
 - `df.rename(index={0:"First"}).head(2)`
 - Rename multiple indices using dictionary:
 - `rename_dict = {0:"first", 1:"second", 2:"third", 4:"fourth"}`
 - `df.rename(index=rename_dict).head()`
-

5. Practical Insights from Jupyter Notebook

- **Inline comments** explain why certain operations fail (e.g., mismatched list lengths).
 - **Warnings** (like FutureWarning) highlight upcoming changes in Pandas behavior.
 - **Custom column insertion (`insert`)** is a neat trick to control column order.
 - **Renaming indices** is useful for readability in reports or presentations.
 - **Outputs in Jupyter** often truncate — use `.head()` and `.tail()` to control visibility.
-

6. Key Takeaways

- Attributes (`index`, `shape`, `columns`, `dtypes`, etc.) help you **inspect structure quickly**.
 - Methods (`info`, `describe`, `head`, `tail`, `transpose`) are essential for **data exploration**.
 - Row/column operations (`add`, `delete`, `rename`) allow **flexible manipulation**.
 - Jupyter Notebook examples add **practical coding context**:
 - Handling warnings.
 - Custom column positioning.
 - Index renaming for clarity.
-

7. Why This Matters

- These are **foundational Pandas operations** for any data analyst.
 - Mastery here ensures you can **clean, explore, and prepare data** before advanced tasks like merging, grouping, or visualization.
 - The Jupyter Notebook perspective makes this knowledge **directly applicable in real coding environments**.
-